

```
// Method which does some processing in the background.
private void backgroundThreadProcessing() {
    [ ... Time consuming operations ... ]
}
```

Using the Handler for Performing GUI Operations

Whenever you're using background threads in a GUI environment it's important to synchronize child threads with the main application (GUI) thread before creating or modifying graphical elements.

Within your application components, Notifications and Intents are always received and handled on the GUI thread. In all other cases, operations that explicitly interact with objects created on the GUI thread (such as Views) or that display messages (like Toasts) must be invoked on the main thread.

If you're running within an Activity, you can also use the `runOnUiThread` method, which lets you force a method to execute on the same Thread as the Activity UI.

```
runOnUiThread(new Runnable() {
    public void run() {
        // TODO Update a View.
    }
});
```

In other circumstances (such as Toasts and Notifications) you can use the Handler class to post methods onto the Thread in which the Handler was created.

Using the Handler class you can post updates to the user interface from a background thread using the Post method. The following code shows the outline for using the Handler to update the GUI thread.

```
// Initialize a handler on the main thread.
private Handler handler = new Handler();

private void mainProcessing() {
    Thread thread = new Thread(null, doBackgroundThreadProcessing,
                                "Background");

    thread.start();
}

private Runnable doBackgroundThreadProcessing = new Runnable()
{
    public void run() {
```